

Q1.

當訂閱的 topic 為 /a/test 時，能不能接收到 topic 為 a/test 的訊息？為什麼？

(hint : 比較兩個 topic 的層數)

Ans: 不能。

因為 topic 是用 "/" 來分層的，/a/test 開頭多了一個 "/"，所以第一層其實是空的，總共有三層（空、a、test），而 a/test 只有兩層（a、test）。沒有用萬用字元的話，每一層都要一樣才會收到，所以訂閱 /a/test 收不到 a/test 的訊息。

實際測試時，pub 到 a/test 的 test_q1 沒有收到，改 pub 到 /a/test 才有收到。



```
Terminal 1 (subscribe)
$ mosquitto_sub -t /a/test -v
/a/test test_q1_compare
^C$

Terminal 2 (publish)
$ mosquitto_pub -t a/test -m test_q1
$ mosquitto_pub -t /a/test -m test_q1_compare
$
```

圖 1 Q1 測試結果

Q2.

Msg format : Place/Floor/Sensor Type/Time/ 共 36 種

- Place : house1、house2
- Floor : firstfloor、secondfloor、roof
- Sensor Type : temp、humid、brightness
- Time : day、night

若只想收到以下內容，那麼 subscriber 的 topic 分別要定為什麼？

1. 所有 roof 在 day 的 brightness
2. house1 firstfloor 的所有內容
3. house2 在 night 的所有內容
4. 所有 roof 的所有內容

Ans:

1. +/roof/brightness/day
2. house1/firstfloor/#
3. house2/+/+/night
4. +/roof/#

+ 只能代表一層，# 可以代表後面所有層，但只能放在最後面。第 1、3 題是中間某幾層不限定，所以用 +；第 2、4 題是要某一層以下的全部內容，所以用 #。

我把 36 種 topic 全部 pub 一次，四個 subscriber 分別收到 2、6、9、12 筆，跟算出來的一樣。

```
Subscriber 1
$ mosquitto_sub -v -t '+/roof/brightness/day'
house1/roof/brightness/day house1-roof-brightness-day
house2/roof/brightness/day house2-roof-brightness-day
^C$

Subscriber 2
$ mosquitto_sub -v -t 'house1/firstfloor/#'
house1/firstfloor/temp/day house1-firstfloor-temp-day
house1/firstfloor/temp/night house1-firstfloor-temp-night
house1/firstfloor/humid/day house1-firstfloor-humid-day
house1/firstfloor/humid/night house1-firstfloor-humid-night
house1/firstfloor/brightness/day house1-firstfloor-brightness-day
house1/firstfloor/brightness/night house1-firstfloor-brightness-night
^C$

Subscriber 3
$ mosquitto_sub -v -t 'house2/+/+/night'
house2/firstfloor/temp/night house2-firstfloor-temp-night
house2/firstfloor/humid/night house2-firstfloor-humid-night
house2/firstfloor/brightness/night house2-firstfloor-brightness-night
house2/secondfloor/temp/night house2-secondfloor-temp-night
house2/secondfloor/humid/night house2-secondfloor-humid-night
house2/secondfloor/brightness/night house2-secondfloor-brightness-night
house2/roof/temp/night house2-roof-temp-night
house2/roof/humid/night house2-roof-humid-night
house2/roof/brightness/night house2-roof-brightness-night
^C$

Subscriber 4
$ mosquitto_sub -v -t '+/roof/#'
house1/roof/temp/day house1-roof-temp-day
house1/roof/temp/night house1-roof-temp-night
house1/roof/humid/day house1-roof-humid-day
house1/roof/humid/night house1-roof-humid-night
house1/roof/brightness/day house1-roof-brightness-day
house1/roof/brightness/night house1-roof-brightness-night
house2/roof/temp/day house2-roof-temp-day
house2/roof/temp/night house2-roof-temp-night
house2/roof/humid/day house2-roof-humid-day
house2/roof/humid/night house2-roof-humid-night
house2/roof/brightness/day house2-roof-brightness-day
house2/roof/brightness/night house2-roof-brightness-night
^C$

Publisher
$ for p in house1 house2; do for f in firstfloor secondfloor roof; do for s in temp humid brightness; do for t in day night; do mosquitto_pub -t "$p/$f/$s/$t" -m "$p-$f-$s-$t"; done; done; done; done; echo "published 36 messages"
published 36 messages
$
```

圖 2 Q2 測試結果

Q3.

排除匿名使用者截圖(參考簡報範例)，帳號設為學號。

先用 `mosquitto_passwd` 建立帳號 (帳號 112652006，密碼 123456)，再到 `mosquitto.conf` 加上 `password_file /etc/mosquitto/passwd` 和 `allow_anonymous false`，然後 restart mosquitto。(我用的 mosquitto 是 2.0 版，要多加一行 `listener 1883` 才會擋匿名。)

之後不加帳密直接 sub 或 pub 都會出現 `not authorised`，密碼打錯也一樣，加上 `-u 112652006 -P 123456` 才能正常收發。

```
Terminal 1 (設定 broker)
$ ls /etc/mosquitto
aclfile.example ca_certificates certs conf.d mosquitto.conf pskfile.example pwfile.example
$ sudo mosquitto_passwd -c /etc/mosquitto/passwd 112652006
Password:
Reenter password:
$ ls /etc/mosquitto
aclfile.example ca_certificates certs conf.d mosquitto.conf passwd pskfile.example pwfile.example
$ sudo chown mosquitto:mosquitto /etc/mosquitto/passwd
$ sudo cat /etc/mosquitto/passwd
112652006:$7$101$fsQ2Vmeqmf93I/Ea$/kYSW06YKH06T/RgI/LroqkFIdIlt/1Q/ghf0YQtdoYwZSpyDZv5SgEdYy0CMNCNXS1LUpFZxe
7Tehvy65sAPg==
$ printf '\nlistener 1883\npassword_file /etc/mosquitto/passwd\nallow_anonymous false\n' | sudo tee -a /etc/
mosquitto/mosquitto.conf > /dev/null
$ cat /etc/mosquitto/mosquitto.conf
# Place your local configuration in /etc/mosquitto/conf.d/
#
# A full description of the configuration file is at
# /usr/share/doc/mosquitto/examples/mosquitto.conf.example

#pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto/

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d

listener 1883
password_file /etc/mosquitto/passwd
allow_anonymous false
$ sudo service mosquitto restart
* Restarting network daemon mosquitto
...done.
$
```

圖 3 建立帳號、修改設定檔並重啟

```
Terminal 2 (subscribe)
$ mosquitto_sub -v -t test
Connection error: Connection Refused: not authorised.
$ mosquitto_sub -v -t test -u 112652006 -P 123456
test hello_from_112652006
^C$

Terminal 3 (publish)
$ mosquitto_pub -t test -m hello
Connection error: Connection Refused: not authorised.
Error: The connection was refused.
$ mosquitto_pub -t test -m hello -u 112652006 -P wrongpw
Connection error: Connection Refused: not authorised.
Error: The connection was refused.
$ mosquitto_pub -t test -m hello_from_112652006 -u 112652006 -P 123456
$
```

圖 4 匿名被拒絕，用帳號密碼可以正常收發

Q4.

Broker : topic、msg	Broker 1 (A) sub 顯示	Broker 2 (B) sub 顯示
A : in、msg1	in msg1	無
A : out、msg2	out msg2	fromA/out msg2

A : out/msg、msg3	out/msg msg3	fromA/msg msg3
A : /out、msg4	/out msg4	無
B : out、msg5	fromB/out msg5	out msg5
B : fromB、msg6	fromB/fromB msg6	fromB msg6

Bridge 設在 A (VM) ，設定如下：

```
connection bridge-01
address <Pi 板 IP>:1883
topic # out 1 out/ fromA/
topic # in 1 fromB/
```

結果討論：

1. A 的 subscriber 訂閱 #，所以 A 自己 pub 的訊息都會照原本的 topic 顯示。
2. out 那行的意思是 A 上 out/ 開頭的訊息會送到 B，而且 out/ 會換成 fromA/，所以 msg3 在 B 變成 fromA/msg。
3. msg2 的 topic 只有 out，因為 # 也會匹配上一層本身，所以 out 也符合 out/#，會被送到 B，只是它沒有 out/ 可以換，就直接在前面加上 fromA/，B 收到的是 fromA/out。
4. msg1 (in) 和 msg4 (/out) 都不符合 out/#，所以只有 A 收得到，B 收不到。
5. in 那行只有設 local-prefix，所以 B 上的所有訊息都會傳回 A，而且前面加上 fromB/，msg5、msg6 在 A 分別變成 fromB/out 和 fromB/fromB，B 自己則是照原本的 topic 顯示。

Q5.

在 MQTT bridge configuration 中，QoS level 有哪幾種？分別描述不同 level 的功能。

Ans: QoS 有 0、1、2 三種。

QoS 0：最多送一次。送出去就不管了，對方也不用回應，所以有可能掉訊息，但速度最快。

QoS 1：至少送一次。對方收到要回 PUBACK，沒收到回應就會重送，所以訊息不會掉，但可能會收到重複的。

QoS 2：剛好送一次。要經過 PUBLISH、PUBREC、PUBREL、PUBCOMP 四次交握，確保不會掉也不會重複，但最慢、負擔也最大。

在 bridge 設定裡，topic 後面的那個數字就是兩個 broker 之間傳這些 topic 時用的 QoS，這次實驗設的是 1。

Q6.

在 `mosquitto.conf` 中，對於設定 `cleansession` 的說明如下：

`cleansession [true | false]`

Set the clean session option for this bridge. Setting to *false* (the default), means that all subscriptions on the remote broker are kept in case of the network connection dropping. If set to *true*, all subscriptions and messages on the remote broker will be cleaned up if the connection drops. Note that setting to *true* may cause a large amount of retained messages to be sent each time the bridge reconnects.

If you are using bridges with `cleansession` set to *false* (the default), then you may get unexpected behavior from incoming topics if you change what topics you are subscribing to. This is because the remote broker keeps the subscription for the old topic. If you have this problem, connect your bridge with `cleansession` set to *true*, then reconnect with `cleansession` set to *false* as normal.

請理解 `cleansession` 參數的目的並回答以下問題：

1. 描述 `true` | `false` 分別的作用。
2. 如果 `cleansession` 設置為 `false`，而且你更改訂閱的主題，會出現什麼意外的行為？為什麼會發生這種情況？該怎麼解決？
3. 什麼是"保留消息 (retained messages)"，為什麼在 `cleansession` 設置為 `true` 時可能會出現大量的保留消息被發送的情況？

Ans:

1. 設成 `true` 的話，bridge 斷線時遠端 broker 會把它的訂閱和還沒送出的訊息都清掉，重新連線後要重新訂閱，斷線期間的訊息就收不到了。設成 `false` (預設) 的話，遠端 broker 會保留 bridge 的訂閱，斷線期間 QoS 1、2 的訊息也會先存起來，重連之後再補送。
2. 如果是 `false`，改了 bridge 的 topic 之後重連，還是會收到舊 topic 的訊息。因為訂閱是存在遠端 broker 上的，重連時會沿用舊的 session，而 bridge 只會去訂閱新的 topic，不會取消舊的，所以舊的訂閱還在。解決方法就是先把 `cleansession` 改成 `true` 重連一次，把舊的清掉，再改回 `false`。
3. retained message 是 publish 時有加 retain 旗標 (例如 `mosquitto_pub -r`) 的訊息，broker 會把每個 topic 最後一筆 retained message 存起來，之後只要有人訂閱這個

topic 就會馬上收到這一筆。設成 true 的話每次重連都要重新訂閱，而每訂閱一次，broker 就會把符合的 retained message 全部再送一次。bridge 通常是訂閱 # 這種範圍很大的 topic，所以只要重連次數一多，就會一直收到一大堆 retained message。

Q7.心得:

這次實驗學到 MQTT 的基本用法。一開始在設定 VM 的時候，因為 VM 沒有關機，網路跟處理器的設定都不能改，關機後才改得了。做 pub/sub 的時候，我一開始把 sub 和 pub 開在同一個 terminal，sub 跑起來之後按 Ctrl+C 跳出來再 pub，當然收不到，後來才知道要開兩個 terminal，一個負責訂閱、一個負責發布。

Q1 原本覺得 /a/test 跟 a/test 應該差不多，實際測了才知道開頭多一個 / 就會多一層。Q2 用 + 和 # 去篩選 topic 還蠻直觀的。bridge 的部分比較複雜，out、in 再加上 prefix 的轉換要想一下，像 msg2 的 topic 是 out 也會被送出去這點一開始沒想到，後來才搞懂。整體來說 MQTT 設定起來不難，pub 跟 sub 只要知道 broker 的 IP 就可以溝通，蠻適合用在 IoT 上的。